

Rapport de Visualisation du fichier Prédiction Weekend

Prédiction du week-end et du jour de la semaine à partir des données de consommation du bâtiment 100276

1. Objectif

Le fichier *prediction_weekend.ipynb* développe un algorithme qui prend en entrée un ligne d'un fichier time series et qui renvoie si ce jour s'agit ou non d'un jour de weekend. Un deuxième bloc de code vise à déterminer le jour précis de la semaine dont il s'agit. Cela nous permet de vérifier si les données de consommation contiennent un signal temporel exploitable (si les habitudes de vie d'un foyer diffèrent entre semaine et weekend).

Les données utilisées proviennent d'un unique bâtiment 100276 sur toute l'année 2018 au pas de temps de 15 minutes.

2. Méthodologie

1ere partie : Prédire si un jour donné est un weekend ou un jour de semaine

Nous commençons par reconstruire, à partir du timestamp de chaque ligne, le nom du jour de la semaine correspondant (*day*), puis nous en déduisons une variable binaire *is.weekend* qui vaut 1 si le jour est un samedi ou un dimanche, et 0 sinon. Cette variable constitue notre cible *y* pour la classification.

La matrice de variables explicatives (*X*) à été construite en sélectionnant l'ensemble des colonnes numériques sans NaN et en retirant la colonne *is.weekend*.

Nous avons ensuite séparé le jeu de données en un ensemble d'entraînement (*X_train*, *y_train*) et un ensemble de test (*X_test*, *y_test*), avec une répartition 70 % contre 30 %, en conservant la même proportion de weekends et de jours de semaine dans les deux sous-ensembles.

Nous avons alors entraîné deux modèles de classification distincts sur ces données : un k plus proches voisins (KNN) avec $k=5$ et un classifieur bayésien naïf gaussien (probabilité d'une classe selon la distribution normale des variables).

Nous avons aussi construit une fonction utilitaire (*predict_if_weekend_KNN* / *predict_if_weekend_GNB*) qui, à partir de l'indice d'une ligne dans *X_test*, retrouve la ligne correspondante dans le jeu de données complet, applique le modèle, et compare la prédiction à la réalité observée.

2eme partie : Identifier précisément le jour de la semaine

Nous avons rajouté 2 colonnes importantes :

- *day.of.year* : attribue un entier de 0 à 365 à tous les jours de l'année
- *day.of.week* : attribue un entier de 0 pour lundi à 6 pour dimanche via un dictionnaire de correspondance pour indiquer le jour de la semaine numériquement.

Cette partie réutilise la première partie, car plutôt que d'essayer de classer directement un jour parmi sept catégories, nous réutilisons notre modèle KNN binaire (weekend / semaine) et l'appliquons successivement à sept observations consécutives, une par jour, en partant d'un jour donné et en avançant de 96 lignes (soit un jour) à chaque fois grâce à la fonction *advanced_n_days_ROW*. Nous obtenons ainsi une séquence de sept prédictions binaires (0 ou 1) couvrant une semaine complète.

Nous cherchons ensuite, dans cette séquence de sept valeurs, la transition caractéristique « weekend (1) suivi d'un jour de semaine (0) », qui signale la fin du weekend et donc le lundi (fonction *find_monday*). Une fois le lundi repéré dans la séquence, il suffit de compter le nombre de jours d'écart entre l'observation initiale et ce lundi pour en déduire le jour de la semaine exact de l'observation initiale (fonction *find_current_day*), par correspondance avec notre dictionnaire qui a un jour associé un indice.

Nous avons du choisir l'heure de la journée à laquelle appliquer cette prédiction (variable *HEURE_CHOISIE*) : nous testons plusieurs heures et retenons celle qui maximise la précision du modèle. Grâce à la analyse des composantes et le coefficient attribué à chaque heure dans le fichier PCA, nous avons pu retenir que 7h maximiserait certainement les résultats de la prédiction.

3. Analyse des résultats

```
Sample Predictions: [0 1 0 0 0 1 0 1 1 0]
Actual Labels:      is.weekend
14558               0
5878                1
16443               0
22332               0
8887                0
6702                1
31596               0
7890                1
5210                1
6380                0
```

In [10]:

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")
```

Model Accuracy: 0.8333

Le modèle KNN entraîné sur la première répartition (70 % contre 30 %) atteint une précision de 83,3 % pour distinguer un week-end d'un jour de semaine à partir d'une seule observation de 15 minutes. Ce résultat confirme que les habitudes de consommation et d'occupation (éclairage, présence, usage des appareils électroménagers) diffèrent suffisamment entre semaine et weekend pour être détectées automatiquement, même sans information temporelle dans les variables d'entrée

```
Number of mislabeled points out of a total  
17520 points : 6888  
Accuracy = 0.6068493150684932
```

Le modèle bayésien naïf gaussien, entraîné sur une répartition différente (50-50 %), atteint une précision plus faible d'environ 60,7 %. Cet écart s'explique en partie par l'hypothèse forte de normalité et d'indépendance conditionnelle des variables que fait ce modèle, hypothèse peu réaliste ici étant donné le grand nombre de variables corrélées entre elles.

```
precise_day_accuracy_test(1000)
```

Accuracy :0.493

Lorsque nous passons à l'identification précise du jour de la semaine (et non plus seulement la distinction entre weekend et semaine), la précision chute nettement. En effet, la recherche du lundi via la séquence de sept prédictions atteint environ 64% de réussite sur un échantillon de 200 tirages aléatoires, et l'identification exacte du jour de la semaine (parmi les sept possibles) atteint environ 49,3% sur 1000 tirages. Cette baisse s'explique par l'effet cumulatif des erreurs : il suffit qu'une seule des sept prédictions binaires de la semaine soit erronée pour que la transition weekend/semaine soit mal repérée, et donc que tout le raisonnement en cascade avec la recherche du lundi le puis calcul du décalage soit erroné.

Nous observons également que l'heure de la journée choisie pour faire la prédiction influence fortement la précision obtenue. Par exemple, une prédiction faite à 7h du matin nous donne cette précision d'environ 50%, contre seulement 30% à midi. Cela suggère que le signal distinguant semaine et weekend est plus marqué tôt le matin qu'en milieu de journée, ce qui confirme les résultats de l'ACP obtenus précédemment.

CONCLUSION

Nous concluons qu'il est possible, à partir des seules variables physiques et comportementales d'un bâtiment, de retrouver une information temporaire avec une assez bonne fiabilité, et de s'en servir pour approcher le jour exact de la semaine de façon plus indirecte et moins fiable.

Cette étude a deux limites majeures. Premièrement, le modèle est entraîné et testé sur un unique bâtiment, donc rien ne garantit que les mêmes performances seraient observables sur un bâtiment aux habitants ou à l'usage différent, ni que les variables les plus discriminantes seraient les mêmes. Deuxièmement, l'approche en cascade pour identifier le jour précis de la semaine amplifie les erreurs du modèle binaire initial, ce qui limite fortement sa fiabilité. Pour améliorer la prédiction, il faudrait imaginer une autre façon de retrouver le bon jour de la semaine, potentiellement en observant les prédictions d'une plage plus large de jours (3 semaines autour de la date au lieu d'une seule, par exemple).

Toutefois, nous retenons que le signal weekend/semaine est bien présent dans les données et donc que les variables comportementales et temporelles sont pertinentes en tant que variables explicatives dans un futur modèle de prédiction de consommation énergétique d'un bâtiment, au même titre que les variables physiques. La variable *is.weekend* pourrait donc être directement intégrée comme variable d'entrée dans un modèle de prédiction de consommation.